Contents

1 Ricerca di soluzioni: *Solving Problems by Searching* **3**
1.1 Dal problema allo spazio degli stati . . . 3
1.1.1 Cammini, soluzioni e costo . . . 3
1.2 Criteri per confrontare gli algoritmi . . . 4
1.3 Ricerca non informata . . . 4
1.3.1 Breadth-first search (BFS) . . . 4
1.3.2 Uniform-cost search (UCS) e best-first . . . 5
1.3.3 Depth-first search (DFS) . . . 5
1.3.4 Profondita' limitata e iterative deepening . . . 5
1.3.5 Ricerca bidirezionale . . . 6
1.4 Ricerca informata: euristiche e A* . . . 6
1.4.1 Ammissibilita' . . . 6
1.4.2 A* search . . . 7
1.5 Ricerca informata con memoria limitata . . . 7
1.5.1 IDA* (iterative-deepening A*) . . . 7
1.5.2 Recursive best-first search (RBFS) . . . 8
1.6 Ricerca locale: cercare una configurazione . . . 8
1.6.1 Hill climbing e i suoi limiti . . . 8
1.6.2 Simulated annealing . . . 9
1.6.3 Beam search ed algoritmi evolutivi . . . 9
1.6.4 Spazi continui . . . 10
1.7 Azioni non deterministiche: piani condizionali . . . 10
1.7.1 Alberi AND-OR . . . 10
1.8 Osservazione incompleta: belief states . . . 11
1.8.1 Ambiente senza osservazioni . . . 11
1.8.2 Osservazione parziale: predict–percept–update . . . 11
1.9 Ricerca online: agire mentre si apprende . . . 12
1.9.1 Strategie online . . . 12
1.9.2 Learning Real-Time A* (LRTA*) . . . 12
1.10 Ricerca avversaria: giochi a somma zero . . . 13
1.10.1 Minimax . . . 13
1.11 Potatura alpha-beta: non esplorare cio' che non puo' servire . . . 14
1.12 Giocare con budget limitato: H-minimax . . . 15
1.12.1 Horizon effect e rimedi . . . 15
1.12.2 Monte Carlo Tree Search . . . 15

1.13 Giochi stocastici e giochi a informazione incompleta 16
1.13.1 Chance nodes ed expectiminimax 16
1.13.2 Informazione nascosta e averaging over clairvoyance 16
1.14 Mappa finale per lo studio 16

1 Ricerca di soluzioni: *Solving Problems by Searching*

Fonte: lezione di Pietro Galliani, **21-Searching.pdf**.

L'idea centrale e' trasformare un problema di pianificazione in un problema di **ricerca di un cammino**. La scelta dell'algoritmo determina quale cammino viene esplorato per primo, e quindi il compromesso tra completezza, ottimalita', tempo e memoria.

1.1 Dal problema allo spazio degli stati

L'astrazione e' appropriata se l'ambiente e' **completamente osservabile, deterministico, statico e noto**, e se e' sufficiente una rappresentazione atomica dello stato.

- le configurazioni dell'ambiente diventano **nodi** (stati);
- le azioni possibili diventano **archi** o transizioni;
- la configurazione iniziale diventa il **nodo iniziale**;
- le configurazioni desiderate diventano i **nodi obiettivo**.

Un problema di ricerca e' dato da

$$\mathcal{P} = (S, s_0, G, \mathrm{Actions}, \mathrm{Result}, \mathrm{ActionCost}),$$

dove:

- S e' l'insieme degli stati possibili;
- $s_0 \in S$ e' lo stato iniziale;
- $G \subseteq S$ e' l'insieme degli stati obiettivo;
- $\mathrm{Actions}(s)$ e' l'insieme delle azioni applicabili in s;
- $\mathrm{Result}(s, a) \in S$, per $a \in \mathrm{Actions}(s)$, e' lo stato successivo;
- $\mathrm{ActionCost}(s, a, s')$ e' il costo di applicare a e raggiungere $s' = \mathrm{Result}(s, a)$.

Il fatto che `Result` restituisca un solo stato formalizza il **determinismo**. Esempi intuitivi sono il Vacuum World e la pianificazione del percorso Arad–Bucharest: non occorre generare l'intero grafo, che puo' essere enorme o persino infinito.

1.1.1 Cammini, soluzioni e costo

Un cammino e' una sequenza

$$(s_0, a_0, s_1, a_1, \dots, a_{n-1}, s_n)$$

per cui, per ogni $i < n$, $a_i \in \mathrm{Actions}(s_i)$ e $s_{i+1} = \mathrm{Result}(s_i, a_i)$. E' una **soluzione** se inoltre $s_n \in G$. Il suo costo e'

$$\mathrm{Cost} = \sum_{i=0}^{n-1} \mathrm{ActionCost}(s_i, a_i, s_{i+1}).$$

Una soluzione e' **ottima** se il suo costo e' minimo fra tutte le soluzioni. Attenzione: il grafo degli stati non coincide con l'albero di ricerca; un nodo dell'albero rappresenta un *cammino* e quindi lo stesso stato puo' comparire in nodi diversi. Bisogna percio' evitare cicli e ripetizioni inutili.

1.2 Criteri per confrontare gli algoritmi

Ogni strategia sceglie un nodo dalla **frontiera** (i nodi generati ma non ancora espansi). La valutiamo chiedendo:

- **Completezza**: se una soluzione esiste, l'algoritmo la trova?
- **Ottimalita'**: la soluzione trovata ha costo minimo?
- **Tempo**: quanti nodi vengono generati/espansi?
- **Spazio**: quanti nodi devono essere mantenuti in memoria?

Useremo b per il branching factor, d per la profondita' della soluzione meno profonda e m per la profondita' massima dell'albero. Le complessita' esponenziali in b spiegano perche' non basta conoscere gli algoritmi: bisogna scegliere una strategia adatta al problema.

1.3 Ricerca non informata

Una ricerca e' **non informata** quando usa solo la definizione del problema, senza una stima di quanto uno stato sia vicino all'obiettivo.

1.3.1 Breadth-first search (BFS)

BFS espande prima tutti i nodi a profondita' 0, poi quelli a profondita' 1, e cosi' via: la frontiera e' una coda FIFO.

- E' completa se esiste una soluzione.
- E' ottima **solo se tutti i costi sono uguali**: minimizza il numero di passi, non il costo in generale.
- Se la soluzione e' a profondita' d, genera

$$1 + b + b^2 + \cdots + b^d = O(b^d)$$

nodi. Tempo e spazio sono entrambi $O(b^d)$.

Il suo problema principale e' la memoria: prima di esaminare il primo nodo di profondita' d, deve gia' conservare una frontiera di ordine b^d.

1.3.2 Uniform-cost search (UCS) e best-first

La UCS e' l'algoritmo di **Dijkstra** visto come ricerca. Invece di espandere il nodo meno profondo, espande quello con minor costo del cammino gia' percorso:

$$g(n) = \text{costo effettivo da } s_0 \text{ al nodo } n.$$

Quindi trova sempre una soluzione ottima (nel quadro della lezione). Puo' pero' essere peggiore di BFS se molti cammini economici non portano all'obiettivo.

La **best-first search** e' la generalizzazione: mantiene la frontiera ordinata secondo un qualunque criterio di valutazione e ne espande il nodo migliore. UCS e A* sono casi particolari di questa idea.

1.3.3 Depth-first search (DFS)

DFS espande sempre il nodo piu' profondo: segue un ramo finche' possibile e poi torna indietro.

- Lo spazio e' $O(bm)$: conserva essenzialmente il ramo corrente e le alternative locali, molto meno della frontiera di BFS.
- Il tempo nel caso peggiore resta $O(b^m)$.
- Non e' ottima e, se lo spazio degli stati e' infinito, puo' non terminare anche quando una soluzione esiste.

1.3.4 Profondita' limitata e iterative deepening

La **depth-limited search** e' DFS con un limite ℓ: i nodi oltre ℓ non vengono espansi. Evita di perdersi in rami infiniti, ma puo' non trovare una soluzione che richiede una profondita' maggiore.

L'**iterative deepening** ripete la ricerca depth-limited con limiti $1, 2, ...$, finche' trova una soluzione. Ripete parte del lavoro, ma la somma resta asintoticamente

$$O(b^1) + O(b^2) + \cdots + O(b^m) = O(b^m).$$

Ha il basso consumo di memoria di DFS e, in pratica, e' spesso la scelta preferibile per una ricerca non informata. Con costi uniformi trova anche una soluzione a numero minimo di passi.

1.3.5 Ricerca bidirezionale

Quando e' possibile invertire le azioni e l'obiettivo e' noto, si puo' cercare contemporaneamente da s_0 in avanti e dal goal all'indietro, fino a incontrarsi a meta'. Con distanza d, si passa idealmente da una crescita dell'ordine di b^d a due esplorazioni dell'ordine di $b^{d/2}$:

$$b^{d/2} + b^{d/2} \ll b^d.$$

Nelle slide si propone di espandere, con una logica best-first, il lato la cui frontiera e' meno profonda. Non e' sempre applicabile: occorre poter generare correttamente i predecessori e gestire il/i goal.

Condizioni e limiti d'esame. E' una ricerca non informata e richiede un goal identificabile con precisione, il modello delle transizioni e la possibilita' di trovare i predecessori di uno stato. Se la soluzione ha profondita' d, tempo e spazio sono entrambi $O(b^{d/2})$. Non e' una buona strategia in un ambiente totalmente ignoto: l'agente non conosce in anticipo il grafo, le transizioni inverse o tutti i nodi da cui iniziare la ricerca all'indietro.

1.4 Ricerca informata: euristiche e A*

Una ricerca e' **informata** se usa un'euristica

$$h : S \to \mathbb{R}_{\geq 0},$$

dove $h(s)$ stima il costo del cammino piu' economico da s a un goal. L'euristica serve a dirigere la ricerca verso stati promettenti e a non esplorare rami irrilevanti.

Possibili fonti di h: intuizione/conoscenza del dominio, **rilassamenti** del problema (per esempio la distanza in linea d'aria invece delle strade), oppure dati in un approccio ibrido.

1.4.1 Ammissibilita'

Sia $h^*(s)$ il vero costo minimo da s a un obiettivo. Un'euristica e' **ammissibile** se e' sempre ottimistica:

$$\forall s \in S \qquad h(s) \leq h^*(s).$$

Un'euristica non ammissibile puo' comunque essere utile e talvolta piu' rapida nella pratica; l'ammissibilita' non e' una garanzia di efficienza, ma permette di dimostrare l'ottimalita' di A*.

1.4.2 A* search

A* e' una best-first search con funzione di valutazione

$$f(n) = g(n) + h(n),$$

dove $g(n)$ e' il costo gia' sostenuto dal nodo iniziale a n, mentre $h(n)$ stima il costo rimanente. Espande il nodo con f piu' piccolo.

- E' completa: se esiste una soluzione, la trova.
- Se h e' ammissibile, e' ottima.
- Il suo limite pratico e' la memoria, poiche' la frontiera best-first puo' crescere molto.

Idea della dimostrazione di ottimalita'. Sia C^* il costo di una soluzione ottima. Se A* restituisse invece una soluzione di costo $C > C^*$, esisterebbe un primo nodo n del cammino ottimo non scelto. Per questo nodo,

$$f(n) = g^*(n) + h(n) \leq g^*(n) + h^*(n) = C^*.$$

Poiche' A* ha scelto una soluzione di costo C prima di n, dovrebbe valere $f(n) > C^*$: contraddizione. Dunque non puo' restituire una soluzione piu' costosa di quella ottima.

> "Ottimo" significa che il cammino finale e' il migliore; non significa che A* lo trovi nel minor tempo possibile. La qualita' concreta di h resta decisiva.

1.5 Ricerca informata con memoria limitata

Quando A* esaurisce la memoria, si rinuncia a conservare tutta la frontiera e si accetta di ripetere del lavoro.

1.5.1 IDA* (iterative-deepening A*)

IDA* esegue DFS imponendo una soglia su $f(n) = g(n) + h(n)$. Se non trova il goal, la nuova soglia diventa il minimo valore di f osservato ma escluso dalla precedente ricerca.

E' molto piu' parsimoniosa in memoria di A*, ma puo' richiedere piu' tempo, soprattutto quando molti nodi condividono lo stesso valore di f.

1.5.2 Recursive best-first search (RBFS)

RBFS segue un'impostazione depth-first, ma ricorda il valore f della migliore alternativa non ancora esplorata. Se il valore del ramo corrente supera tale alternativa, fa backtracking e prova l'altro ramo.

Puo' essere piu' efficiente di IDA*, in particolare con euristiche poco informative, ma puo' anche cambiare ripetutamente decisione e riesplorare porzioni dell'albero. IDA*, RBFS e A* formano quindi una famiglia di scelte nel compromesso **memoria contro tempo**.

1.6 Ricerca locale: cercare una configurazione

La ricerca dei capitoli precedenti cerca un **piano**, cioe' una sequenza di azioni. Alcuni problemi, invece, chiedono direttamente una configurazione che soddisfi un vincolo o ottimizzi un criterio: nel problema delle n regine, per esempio, interessa una disposizione della scacchiera, non la sequenza con cui le regine sono state collocate.

Nella **ricerca locale** gli stati sono configurazioni vicine tra loro e si massimizza una funzione obiettivo $V : S \to \mathbb{R}$. Se $c(s)$ e' il numero di conflitti, un possibile obiettivo per le n regine e' $V(s) = -c(s)$. Di solito si conserva solo la configurazione corrente, non l'intero cammino: si risparmia memoria, ma non si ottiene automaticamente una soluzione ottima.

1.6.1 Hill climbing e i suoi limiti

Da uno stato iniziale, spesso scelto casualmente, l'**hill climbing** sceglie il vicino s' che migliora di piu' l'obiettivo e ripete finche' non esiste un miglioramento:

$$s \leftarrow \arg\max_{s' \in N(s)} V(s') \qquad \text{se } V(s') > V(s).$$

E' semplice e utile anche come componente di algoritmi piu' complessi, ma puo' arrestarsi in:

- un **massimo locale**, migliore dei soli vicini ma non globale;
- un **plateau**, regione in cui i vicini hanno lo stesso valore;
- una **cresta** (*ridge*), lungo la quale un miglioramento richiede una sequenza di mosse non immediatamente vantaggiose.

Varianti importanti:

- **sideways moves**: permette mosse con valore invariato, per attraversare un plateau;
- **stochastic hill climbing**: sceglie casualmente fra vicini buoni;
- **first-choice**: prova vicini casuali finche' trova il primo miglioramento;

- **random-restart**: dopo un blocco ricomincia da una configurazione casuale e restituisce il miglior stato visto in tutti i tentativi.

1.6.2 Simulated annealing

Il **simulated annealing** corregge il difetto essenziale di hill climbing: a volte accetta intenzionalmente una mossa peggiorativa, cosi' puo' uscire da un massimo locale. Se $\Delta = V(s') - V(s)$, una regola tipica e'

$$\Pr(\text{accettare } s') = \begin{cases} 1 & \text{se } \Delta \geq 0, \\ e^{\Delta/T} & \text{se } \Delta < 0, \end{cases}$$

dove $T > 0$ e' la **temperatura**. All'inizio T e' alta, quindi anche mosse peggiori sono abbastanza probabili e la ricerca esplora. Riducendo gradualmente T, l'algoritmo diventa sempre piu' simile a hill climbing: sfrutta le zone gia' promettenti.

1.6.3 Beam search ed algoritmi evolutivi

La **beam search** mantiene $k > 1$ stati anziche' uno solo. A ogni passo genera tutti i loro vicini e conserva i k migliori. Questo fa comunicare informazioni tra i diversi filoni di ricerca; se pero' gli stati diventano troppo simili, si riduce a un hill climbing piu' costoso. Anche qui esistono versioni stocastiche.

Un **algoritmo evolutivo** mantiene una popolazione di configurazioni, valutata da una funzione di fitness. Il ciclo e':

1. genera una popolazione iniziale casuale;
2. seleziona le configurazioni migliori come genitori;
3. applica **crossover** per produrre figli;
4. applica con piccola probabilita' una **mutazione** casuale;
5. opzionalmente conserva direttamente i migliori genitori (**elitismo**).

L'intuizione e' combinare parti buone di soluzioni diverse. La scelta della rappresentazione e dell'operatore di crossover e' decisiva: deve poter preservare contributi utili dei genitori, non solo mescolarli senza significato.

1.6.4 Spazi continui

Se uno stato e' un vettore $x = (x_1, \dots, x_n) \in \mathbb{R}^n$ e l'obiettivo $f(x)$ e' sufficientemente regolare, si possono usare strumenti di analisi, come gradient descent/ascent o Newton–Raphson. In presenza di vincoli regolari si entra nell'ottimizzazione vincolata; il caso lineare e' la programmazione lineare, risolvibile in tempo polinomiale (per esempio con il metodo del simplesso).

1.7 Azioni non deterministiche: piani condizionali

Se un'azione non determina un unico stato successivo, la funzione di transizione diventa

$$\mathrm{RESULTS}(s, a) \subseteq S, \qquad \mathrm{RESULTS}(s, a) \neq \emptyset.$$

Non basta piu' scegliere una sola sequenza di azioni: il piano deve specificare cosa fare per ogni possibile esito. Per esempio, un'azione `suck` nel Vacuum World puo' condurre a stati diversi; un piano puo' avere la forma

$$[\texttt{suck},\ \text{if state} = 5 \text{ then } [\texttt{right}, \texttt{suck}] \text{ else } []].$$

1.7.1 Alberi AND-OR

Gli **OR-node** rappresentano la scelta dell'agente: per risolverli e' sufficiente scegliere almeno un successore risolvibile. Gli **AND-node** rappresentano gli esiti dell'ambiente: sono risolti solo se *tutti* i loro successori sono risolvibili.

Formalmente, una soluzione e' un sottoalbero tale che:

1. contiene la radice;
2. ogni sua foglia e' un goal;
3. per ogni OR-node contiene almeno un successore;
4. per ogni AND-node contiene tutti i successori.

Le strategie deterministiche (DFS, BFS, euristiche, A*) si possono adattare. In un AND-node si costruisce un piano per ogni esito e li si combina in un piano condizionale:

$$[\text{if } s_1 \text{ then } p_1 \text{ else if } s_2 \text{ then } p_2 \ \cdots \ \text{else if } s_k \text{ then } p_k].$$

Gli alberi possono essere infiniti; durante la ricerca i cicli vengono trattati come fallimenti del ramo, poiche' un buon piano non ha bisogno di attraversarli.

1.8 Osservazione incompleta: belief states

Quando l'agente non conosce esattamente lo stato fisico, ragiona su un **belief state** $b \subseteq S$: l'insieme degli stati che ritiene possibili. Questo trasforma l'incertezza in uno spazio di ricerca ordinario, ma piu' grande.

1.8.1 Ambiente senza osservazioni

Nel caso estremo, l'agente e' senza sensori e deve trovare una sequenza che funzioni indipendentemente dallo stato iniziale reale. La formulazione in belief states e':

- stati: sottoinsiemi $b \subseteq S$;
- stato iniziale: l'insieme degli stati fisici inizialmente possibili;
- azioni legali (assumendo che un'azione illegale non faccia nulla):

$$\mathrm{Actions}(b) = \bigcup_{s \in b} \mathrm{Actions}(s);$$

- transizione deterministica:

$$\mathrm{Result}(b, a) = \{s' \mid \exists s \in b,\ s' = \mathrm{Result}(s, a)\};$$

- goal: b e' un belief goal se $b \subseteq G$.

Con N stati fisici esistono fino a 2^N belief states. Non tutti sono raggiungibili, e si puo' fare pruning: se un piano risolve b, allora risolve anche ogni $b' \subseteq b$. Per problemi grandi, rappresentare esplicitamente un insieme di stati puo' essere impraticabile; una rappresentazione logica puo' essere piu' compatta.

1.8.2 Osservazione parziale: predict–percept–update

Con osservazioni parziali, dopo un'azione il percetto ricevuto restringe il belief state. Dato b e a, il ciclo e':

$$\begin{aligned}
\mathrm{Predict}(b, a) &= \hat{b} = \{\mathrm{Result}(s, a) \mid s \in b\},\\
\mathrm{PossiblePercepts}(\hat{b}) &= \{o \mid \exists s \in \hat{b},\ o = \mathrm{Percept}(s)\},\\
\mathrm{Update}(\hat{b}, o) &= \{s \in \hat{b} \mid \mathrm{Percept}(s) = o\}.
\end{aligned}$$

L'azione diventa quindi non deterministica a livello dei belief states, perche' l'agente non sa quale percetto ricevera':

$$\text{RESULTS}(b, a) = \{\text{Update}(\text{Predict}(b, a), o) \mid o \in \text{PossiblePercepts}(\text{Predict}(b, a))\}.$$

Si puo' dunque applicare una AND-OR search: l'agente sceglie l'azione, mentre l'ambiente determina il percetto/esito.

1.9 Ricerca online: agire mentre si apprende

Finora l'agente calcolava l'intero piano e poi agiva. Nella **ricerca online** l'ambiente e' ignoto: l'agente deve alternare computazione, percezione e azione. Nel modello della lezione l'ambiente e' deterministico e osservabile, ma il costo $c(s, a, s')$ e lo stato risultante s' sono noti solo dopo aver provato l'azione.

La qualita' si misura con il **competitive ratio**:

$$\frac{\text{costo totale del cammino percorso}}{\text{costo del cammino ottimo}}.$$

L'inefficienza non e' eliminabile: anche se tutte le azioni sono reversibili, un ambiente sconosciuto puo' costringere qualunque agente online a esplorazioni arbitrariamente costose. Se esistono azioni irreversibili, l'agente puo' inoltre entrare in un dead end da cui non e' possibile uscire.

1.9.1 Strategie online

- **Online DFS**: prova azioni non ancora esplorate e torna indietro quando serve. Se le azioni sono reversibili e il goal e' raggiungibile, lo trova prima o poi; il suo competitive ratio puo' pero' essere arbitrariamente cattivo.
- **Random walk**: sceglie casualmente un'azione non provata, se disponibile, altrimenti una qualunque azione. Con azioni reversibili raggiunge eventualmente il goal raggiungibile, ma il costo puo' essere esponenziale.
- **Hill climbing online**: un'euristica puo' guidarlo verso il goal, ma non puo' fare random restart quando incontra un minimo locale, perche' non puo' ricominciare liberamente dall'ambiente reale.

1.9.2 Learning Real-Time A* (LRTA*)

LRTA* unisce hill climbing e memoria. Mantiene una tabella $H(s)$, inizializzata con l'euristica $h(s)$, che rappresenta la migliore stima corrente del costo da s al goal. Dopo aver osservato la transizione eseguita, aggiorna lo stato precedente:

$$H(s) \leftarrow \min_{a \in \text{Actions}(s)} \text{LRTA}^*\text{-Cost}(s, a, \text{result}[s, a], H),$$

dove $\text{LRTA}^*\text{-Cost}(s, a, s', H)$ restituisce $h(s')$ se il successore s' non e' ancora noto; altrimenti restituisce $\text{ActionCost}(s, a, s') + H(s')$.

Poi sceglie l'azione che minimizza lo stesso costo stimato. Aggiornando H, LRTA* impara che certe zone apparentemente buone sono vicoli ciechi o minimi locali e puo' quindi uscirne.

1.10 Ricerca avversaria: giochi a somma zero

Finora l'ambiente non aveva obiettivi propri. Nei giochi, altri agenti scelgono azioni che possono essere in conflitto con quelle dell'agente. Il caso base e' un gioco a due giocatori, deterministico, a informazione perfetta, a lunghezza finita e **a somma zero**: cio' che e' un guadagno per MAX e' una perdita per MIN.

Un modello di gioco contiene:

- uno stato iniziale s_0;
- $\text{TO-MOVE}(s)$, il giocatore che deve muovere in s;
- $\text{ACTIONS}(s)$, le mosse legali;
- $\text{RESULT}(s, a)$, lo stato dopo la mossa a;
- $\text{IS-TERMINAL}(s)$, che riconosce la fine della partita;
- $\text{UTILITY}(s)$, il punteggio finale di uno stato terminale.

1.10.1 Minimax

Il valore minimax di uno stato e' il miglior risultato che MAX puo' **garantirsi**, assumendo che anche MIN giochi in modo perfetto. Si calcola dal basso verso l'alto:

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s), & \text{se IS-TERMINAL}(s), \\ \max_{a \in \text{ACTIONS}(s)} \text{MINIMAX}(\text{RESULT}(s, a)), & \text{se TO-MOVE}(s) = \text{MAX}, \\ \min_{a \in \text{ACTIONS}(s)} \text{MINIMAX}(\text{RESULT}(s, a)), & \text{se TO-MOVE}(s) = \text{MIN}. \end{cases}$$

MAX sceglie dunque una mossa che massimizza il valore, mentre MIN ne sceglie una che lo minimizza. Con branching factor medio b e profondita' media m, minimax richiede tempo $O(b^m)$: per gli scacchi,

dove approssimativamente $b \approx 35$ e $m \approx 80$, la ricerca esaustiva e' impraticabile. Lo spazio e' $O(bm)$, riducibile a $O(m)$ con una visita depth-first accurata.

Pseudocodice da saper ricostruire. La funzione principale restituisce la mossa con valore maggiore; le due funzioni ricorsive restituiscono la coppia `(valore, mossa)`:

```
MINIMAX(pos):
    valore, mossa = MAX-VALUE(pos)
    restituisci mossa

MAX-VALUE(pos):
    se TERMINAL(pos): restituisci (UTILITY(pos), null)
    migliore = (-infinito, null)
    per ogni mossa in ACTIONS(pos):
        valore, _ = MIN-VALUE(RESULT(pos, mossa))
        se valore > migliore.valore: migliore = (valore, mossa)
    restituisci migliore

MIN-VALUE(pos):
    se TERMINAL(pos): restituisci (UTILITY(pos), null)
    migliore = (+infinito, null)
    per ogni mossa in ACTIONS(pos):
        valore, _ = MAX-VALUE(RESULT(pos, mossa))
        se valore < migliore.valore: migliore = (valore, mossa)
    restituisci migliore
```

Con piu' di due giocatori si puo' usare una funzione $\mathrm{UTILITY}(s, p)$ per ciascun giocatore p e mantenere i punteggi di tutti. La stessa idea si estende, ma possono emergere alleanze tra giocatori.

1.11 Potatura alpha-beta: non esplorare cio' che non puo' servire

Alpha-beta produce la **stessa scelta di minimax**, ma evita sottoalberi che non possono modificare la decisione. Durante l'esplorazione di un nodo si mantengono due limiti:

- α: il valore piu' alto che MAX puo' gia' garantire;
- β: il valore piu' basso che MIN puo' gia' garantire.

Inizialmente $[\alpha, \beta] = [-\infty, +\infty]$. In un MAX-node si aggiorna α con il massimo valore visto; in un MIN-node si aggiorna β con il minimo valore visto. Quando $\alpha \geq \beta$, il resto del sottoalbero puo' essere scartato: il giocatore che muove non lo sceglierebbe comunque. Questa e' una **potatura**, non un'approssimazione.

L'ordine delle mosse e' cruciale:

- con ordine ideale (prima le mosse migliori), il tempo scende a $O(b^{m/2})$;
- con ordine casuale, nelle slide la stima tipica e' $O(b^{3m/4})$;

- criteri specifici del gioco o dati di partite possono suggerire mosse promettenti e migliorare la potatura.

1.12 Giocare con budget limitato: H-minimax

Anche alpha-beta deve raggiungere posizioni terminali, e in giochi grandi spesso non e' possibile. Si fissa allora una profondita' di cutoff e si valuta la posizione non terminale con una funzione euristica $\mathrm{EVAL}(s)$, imponendo $\mathrm{EVAL}(s) = \mathrm{UTILITY}(s)$ per gli stati terminali.

La ricorrenza di **H-minimax** e' minimax, eccetto per il caso di cutoff:

$$\text{H-MINIMAX}(s, d) = \mathrm{EVAL}(s) \qquad \text{se IS-CUTOFF}(s, d).$$

Negli scacchi, una euristica base puo' sommare il valore dei pezzi:

$$\mathrm{EVAL}(s) = \#\text{pedoni} + 3\#\text{cavalli} + 3\#\text{alfieri} + 5\#\text{torri} + 9\#\text{regine}.$$

E' solo una stima: il valore di un pezzo dipende dalla posizione e dalla fase della partita.

1.12.1 Horizon effect e rimedi

Un cutoff puramente per profondita' puo' provocare l'**horizon effect**: se una perdita inevitabile cade appena oltre l'orizzonte di ricerca, il programma preferisce mosse che la rimandano oltre il cutoff, anche quando peggiorano ulteriormente la posizione.

Due rimedi presentati sono:

- interrompere la ricerca solo in posizioni **quiescenti**, cioe' senza mosse imminenti che cambierebbero molto la valutazione;
- usare **singular extensions**: oltre il cutoff si esplorano soltanto mosse chiaramente migliori delle alternative.

Si possono inoltre fare **forward pruning** (considerare solo le mosse piu' promettenti, oppure approfondire meno le altre) e usare **lookup tables** per aperture, finali o posizioni ricorrenti gia' risolte.

1.12.2 Monte Carlo Tree Search

Quando b e' enorme, come nel Go, esplorare tutte le mosse anche a profondita' limitata puo' essere inefficace. La **Monte Carlo Tree Search** esegue molte simulazioni guidate da una distribuzione di

mosse (casuale, appresa dai dati o dal self-play) e sceglie la mossa con la migliore percentuale di vittorie stimata. L'idea e' esplorare poche alternative in profondita', invece di moltissime alternative in modo superficiale.

1.13 Giochi stocastici e giochi a informazione incompleta

1.13.1 Chance nodes ed expectiminimax

Nei giochi con casualita', come backgammon o Yahtzee, si aggiunge un giocatore fittizio **CHANCE** (o Nature) che sceglie un esito r con probabilita' $P(r)$. Non e' un avversario: il suo nodo calcola un valore atteso. La ricorrenza di **expectiminimax** conserva i casi MAX e MIN di minimax e aggiunge:

$$\text{EXMINIMAX}(s) = \sum_{r \in \text{ACTIONS}(s)} P(r) \cdot \text{EXMINIMAX}(\text{RESULT}(s, r)) \qquad \text{se TO-MOVE}(s) = \text{CHANCE}.$$

Le idee di alpha-beta, euristiche e forward pruning si applicano anche qui. Per potare nodi casuali servono limiti superiore e inferiore dell'utilita' attesa; inoltre non conta solo l'ordine degli esiti, ma anche la loro probabilita'. La casualita' ingrandisce fortemente l'albero: due dadi, per esempio, moltiplicano per 36 il numero di esiti a ogni turno.

1.13.2 Informazione nascosta e averaging over clairvoyance

Nei giochi di carte lo stato non e' completamente osservabile. Un'approssimazione economica e' l'**averaging over clairvoyance**: si risolve il gioco assumendo, separatamente, ciascuno stato reale possibile (come se l'agente conoscesse le carte), poi si media il valore con expectiminimax.

Il limite e' fondamentale: questo metodo non rappresenta lo **stato informativo** del giocatore. Per esempio, non distingue il caso in cui un premio e' a sinistra da quello in cui e' a destra, se il giocatore non puo' osservarlo; per lo stesso motivo non modella bene il bluff. Puo' essere utile come approssimazione, ma il modello corretto usa belief states, al costo di una crescita molto forte di tempo e memoria.

1.14 Mappa finale per lo studio

1. **Spazio degli stati**: modella stati, azioni, transizione e costi; scegli poi la strategia di esplorazione.

2. **Ricerca classica**: senza euristica, iterative deepening e' spesso il compromesso pratico; con un'euristica ammissibile, A* e' ottima. Se la memoria e' il limite, usa IDA* o RBFS.
3. **Ricerca locale**: cerca una configurazione, non un piano. Hill climbing e' economico ma puo' bloccarsi; simulated annealing, beam search ed evoluzione introducono esplorazione e diversita'.
4. **Azioni non deterministiche**: la soluzione e' un piano condizionale; gli OR-node sono scelte dell'agente, gli AND-node sono esiti che vanno tutti gestiti.
5. **Informazione incompleta**: sostituisci lo stato fisico con un belief state. Con osservazioni parziali applica il ciclo predict–percept–update e una ricerca AND-OR.
6. **Ambiente ignoto**: nella ricerca online si agisce mentre si apprende; l'inefficienza e' inevitabile. LRTA* aggiorna l'euristica memorizzata per evitare di insistere sui minimi locali.
7. **Giochi avversari**: minimax sceglie il massimo valore garantibile contro MIN; alpha-beta conserva la stessa scelta eliminando rami irrilevanti.
8. **Giochi complessi**: H-minimax usa una valutazione euristica al cutoff; expectiminimax sostituisce ai nodi casuali la media pesata degli esiti. Con informazione nascosta, il riferimento corretto resta il belief state.